AdaBoost Hyperparameters and Tuning

Contents

1. Overview . . . 2
2. The hyperparameters of AdaBoostClassifier . . . 3
   - 2.1 estimator: the weak learner . . . 4
   - 2.2 n_estimators: the number of stages . . . 5
   - 2.3 learning_rate: the say of every stump, scaled . . . 5
   - 2.4 algorithm: removed . . . 5
3. n_estimators: from underfitting to overfitting . . . 6
4. learning_rate: shrinkage . . . 7
   - 4.1 What the learning rate does . . . 7
   - 4.2 Why a smaller alpha slows learning . . . 8
   - 4.3 The trade-off with n_estimators . . . 9
5. Tuning with GridSearchCV . . . 11
6. Summary . . . 13
7. Sources . . . 13
8. Key terms . . . 14

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate), 10 (Tune). See the Course map.



- **Builds on:** Learning rate; Cross-validation; Hyperparameter tuning; Grid and random search; AdaBoost.
- **Compare with:** Train-test split; OOB score; Gradient boosting; Bayesian optimisation; Optuna; Keras Tuner.

1. Overview

Key point: AdaBoost has two settings that matter, `n_estimators` (how many stumps) and `learning_rate` (how much say each stump gets); they trade off against each other, so a **grid search** (G-872) tunes them together.

AdaBoost is boosting with stumps: small models trained one after another, each focusing on the mistakes of the ones before (see stage by stage). AdaBoost has few **hyperparameters** (G-910), settings we choose before training.

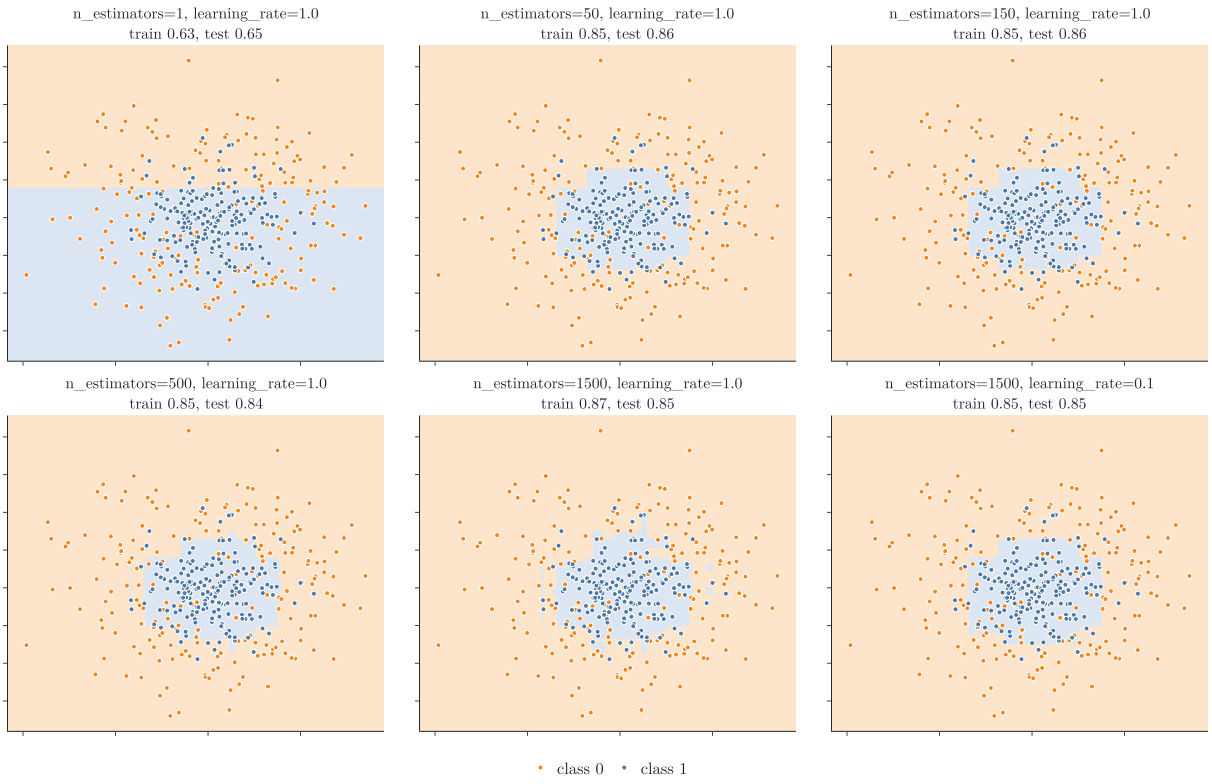


Figure 1: Decision surfaces of AdaBoost on noisy circles: 1 to 1,500 stumps with learning_rate 1.0, and 1,500 stumps with learning_rate 0.1. Titles give training and test accuracy

Gradient boosting and XGBoost, covered in later Notes, have many hyperparameters. AdaBoost is simpler: scikit-learn's `AdaBoostClassifier` has only three, plus `random_state`, which only makes the results repeatable.

The present Note:

- covers what each hyperparameter does (section 2);
- shows `n_estimators` and `learning_rate` at work on **decision surfaces** (G-560) (Figure 1, sections 3 and 4);
- tunes both together with `GridSearchCV` (section 5).

The Notebook (`ML-112-adaboost-hyperparameters.ipynb`) runs every experiment. The Dash app `app.py` has a control for each hyperparameter and redraws the decision surface.

2. The hyperparameters of AdaBoostClassifier

> **Key point:** `estimator` is the **weak learner** (G-2104) (a stump by default), `n_estimators` the number of stages (50 by default), `learning_rate` a multiplier on every alpha (1.0 by default).

Figure 2 marks where each one acts on the boosting loop. Watch the three orange boxes: one picks the model inside the loop, one scales each model's say, and one caps how many times the loop runs.

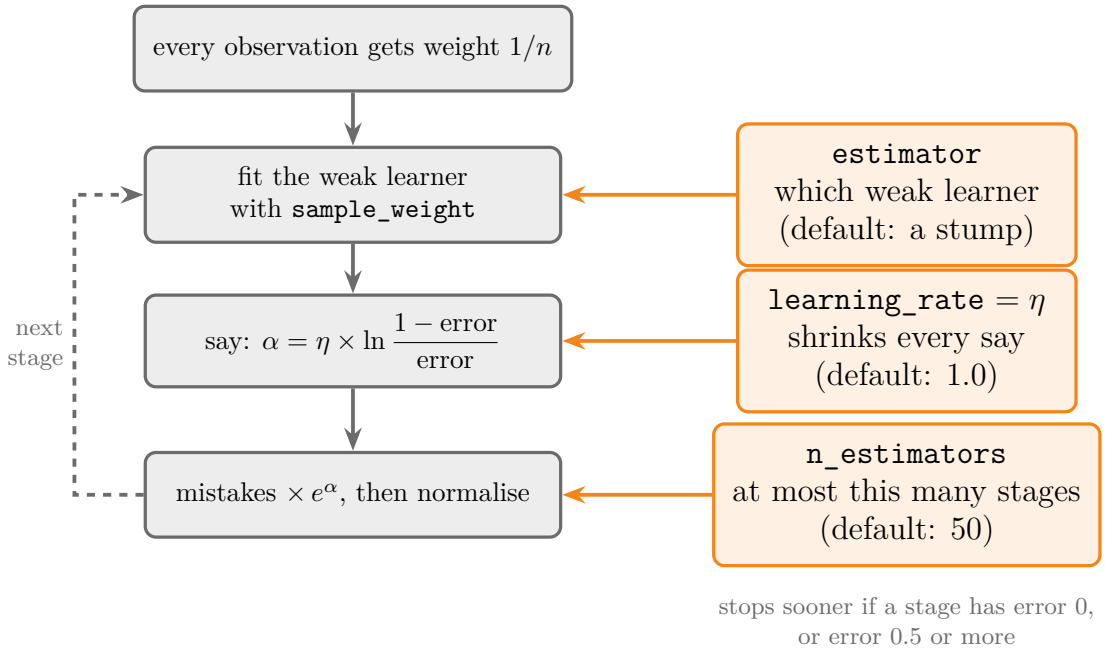


Figure 2: The scikit-learn AdaBoost loop (SAMME) and the three hyperparameters: estimator sets the weak learner, learning_rate scales every alpha, n_estimators caps the number of stages

2.1 estimator: the weak learner

> **Key point:** Any classifier that accepts sample weights can be boosted; in practice we almost always keep the default, a decision stump.

`estimator` is the base model that AdaBoost trains at every stage, the **weak learner** (G-2104). If we leave it out, it is `DecisionTreeClassifier(max_depth=1)`, a **decision stump** (G-559; a tree with one question, see **decision stumps** (G-559)).

Other algorithms are allowed, such as logistic regression or an SVM, with one condition: the model's `fit` must accept a `sample_weight` argument, because that is how scikit-learn passes the observation weights to it (weights instead of upsampling). KNN has no such argument, so AdaBoost refuses it:

Python: KNN cannot be boosted.

```python
from sklearn.ensemble import AdaBoostClassifier
from sklearn.neighbors import KNeighborsClassifier

AdaBoostClassifier(KNeighborsClassifier()).fit(X, y)
```

```
# ValueError: KNeighborsClassifier doesn't
# support sample_weight.
```

In practice, decision trees are used almost every time, and stumps are the default (section 5 tests deeper trees). The `estimator` setting is one we rarely change.

> **Extra:** Older code calls this parameter `base_estimator`, a name since removed (see the bagging classifier in code), so `AdaBoostClassifier(base_estimator=...)` now fails; passing the model as the first argument works in every version.

2.2 n_estimators: the number of stages

> **Key point:** The maximum number of **weak learners** (G-2104). Training stops earlier if a stump classifies the training data perfectly, or if a new stump is no better than guessing.

`n_estimators` (G-1290) is the number of base models, one per stage. The number is a maximum. If some stump makes no mistakes on its weighted data, boosting stops there, because there is nothing left to fix (a stump with no mistakes). Boosting also stops if a new stump's weighted error reaches 0.5 or more on two classes, no better than guessing; that stump is thrown away (scikit-learn source). `n_estimators` is the most important **hyperparameter** (G-910) of AdaBoost; section 3 shows its effect.

2.3 learning_rate: the say of every stump, scaled

> **Key point:** Every alpha is multiplied by the learning rate. Below 1, each stump's say shrinks and the weights change less at each stage: learning slows down.

`learning_rate` (G-1069) is a number multiplied into every stump's weight. Section 4 explains it in full.

2.4 algorithm: removed

> **Key point:** Old versions offered two algorithms, SAMME and SAMME.R; current scikit-learn has only SAMME and no `algorithm` parameter.

Older versions of `AdaBoostClassifier` had a fourth **hyperparameter** (G-910), `algorithm`, with two choices: `"SAMME"` and `"SAMME.R"`. SAMME.R, which used predicted probabilities instead of hard votes, was the default and typically converged faster (scikit-learn 1.5 docs).

> **Extra:** The `algorithm` parameter has since been removed. scikit-learn 1.9, which this Note uses, accepts only `estimator`, `n_estimators`, `learning_rate` and `random_state`, and always runs SAMME. Code that passes `algorithm="SAMME.R"` fails with a `TypeError`; deleting the argument fixes it.

3. n_estimators: from underfitting to overfitting

> **Key point:** One stump underfits; tens of stumps fit the circle; very many start drawing small islands around single points, the sign of overfitting.

The data is the noisy concentric circles of seeing it in classification: 500 **observations** (G-1374; points, one row of the data table each), 2 **features** (G-772; input variables, the two coordinates) and a **target** (G-1949) class: a small blue disc inside an orange ring, with so much noise that the classes overlap. We train on 400 points and test on 100.

With the default settings (50 stumps, learning rate 1.0), 10-fold **cross-validation** (G-510; the data is split into 10 parts and each part in turn is used for testing, see cross-validation with a pipeline) gives an accuracy of **0.812**. Figure 1 shows what changing `n_estimators` does. Each **decision surface** (G-560), the predicted class at every point of the plane, is drawn by predicting a grid of points, as in **decision surfaces** (G-560).

A test set of 100 points is noisy: one point moves the accuracy by 0.01. So the table averages 20 fresh circles datasets of 400 training points, each tested on 5,000 new points from the same distribution. Figure 1's titles show the single dataset drawn.

n_estimators	Surface (Figure 1)	Train	Test
1	one straight cut: underfitting	0.631	0.611
50	a blocky disc, close to the true shape	0.879	**0.846**
150	about the same disc	0.883	0.845
500	small notches appear around single points	0.887	0.844
1,500	isolated strips and boxes: overfitting	0.911	0.837

- **Too few stumps underfit.** **Underfitting** (G-2035) means the model is too simple for the pattern. One stump can only cut the plane once; it cannot draw a disc.
- **Too many stumps overfit.** **Overfitting** (G-1429) means the model fits the noise of the training data. Each extra stage focuses on the observations still misclassified, and on noisy data those are mostly noise. From 50 to 1,500 stumps the training accuracy climbs from 0.879 to 0.911 while the test accuracy slips from 0.846 to 0.837: the gap between them more than doubles. Experiments with noisy labels show the same: AdaBoost puts large weights on the noisy observations and its accuracy drops (Dietterich 2000). With 1,500 stumps, the surface grows small regions around single points.
- **Training time grows** with the number of stumps: 1,500 stumps take 30 times as long to train as 50.

Figure 3 grows the two 1,500-stump models of Figure 1 (bottom middle and bottom right) stump by stump, on the same split. Watch the left panel after 500 stumps: notches and thin strips appear around single points, while learning rate 0.1 on the right keeps a smooth blocky disc.

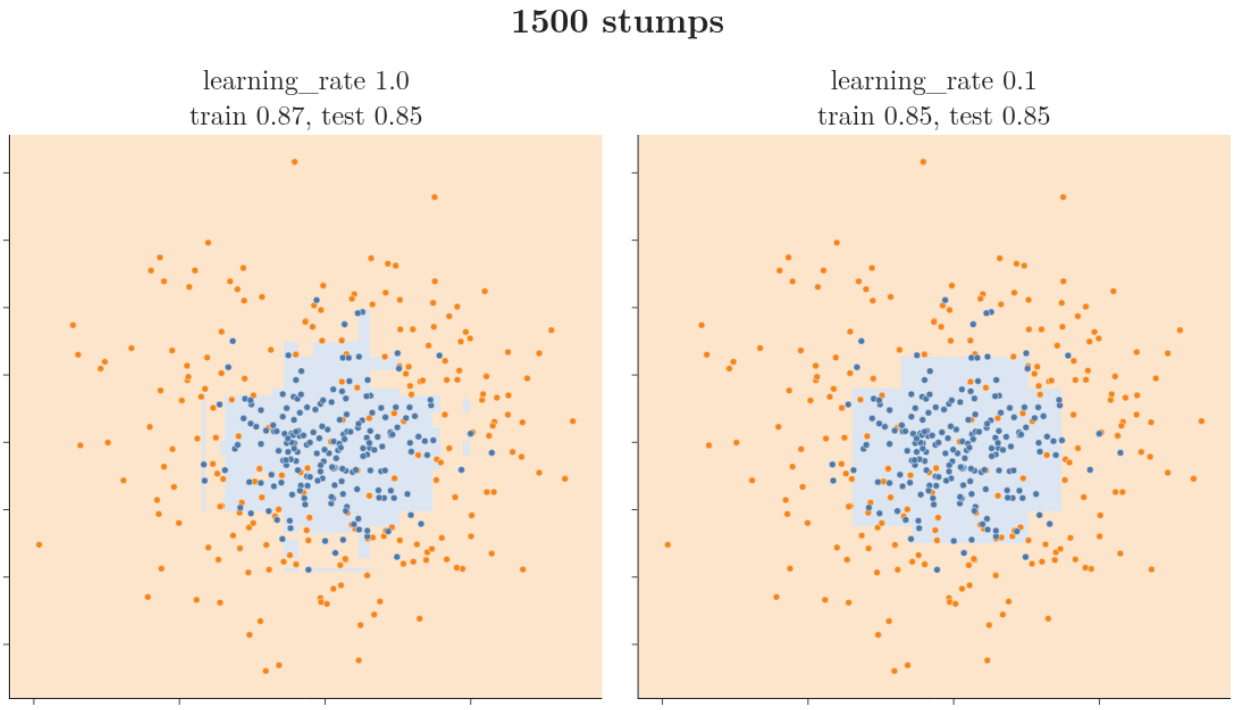


Figure 3: The decision surface after 1 to 1,500 stumps, learning rate 1.0 (left) and 0.1 (right), on the training split; titles give training and test accuracy

So `n_estimators` needs a middle value, like `max_depth` for a decision tree (**max_depth** (G-1184)).

> **Extra:** On this data the overfitting of stumps is slow: 1,500 stumps lose less than 0.01 of test accuracy. With deeper base trees it is much faster: section 5 shows 50 trees of depth 8 reaching training accuracy 1.00 while the test accuracy falls to 0.813.

4. learning_rate: shrinkage

> **Key point:** The learning rate multiplies every alpha. A smaller alpha means smaller weight updates, so each stage changes less, learning is slower, and **overfitting** (G-1429) is slower too.

4.1 What the learning rate does

> **Key point:** alpha becomes learning_rate times the usual alpha; at the default 1.0 nothing changes.

1. **In words:** each stump's say is the usual alpha (α, the say of a stump; see the say of the stump) times the learning rate η (eta).
2. **Formula:**

$$\alpha_t = \eta \times \frac{1}{2} \ln\left(\frac{1 - \text{error}_t}{\text{error}_t}\right)$$

3. **Example:** a stump with error 0.3 has the usual alpha 0.4236 (stage 1). With $\eta = 0.1$:

$$\alpha = 0.1 \times 0.4236 = 0.0424$$

In the Notebook, the first stump's weight on the circles data is 0.5322 with $\eta = 1.0$ and 0.0532 with $\eta = 0.1$: exactly ten times smaller.

> **Extra:** scikit-learn's SAMME alpha has no factor 1/2 (weights instead of upsampling), so its first-stump weight 0.5322 corresponds to our 0.2661. The learning rate multiplies whichever version is used.

4.2 Why a smaller alpha slows learning

> **Key point:** The weight update uses e^{α} and $e^{-\alpha}$; a small alpha keeps both close to 1, so the weights barely move at each stage.

The weights are updated with e^{α} for mistakes and $e^{-\alpha}$ for correct observations (updating the weights). With a smaller alpha, both factors stay closer to 1: the update has a smaller **amplitude**. For our error-0.3 stump:

learning_rate	alpha	mistakes $\times e^{\alpha}$	correct $\times e^{-\alpha}$
1.0	0.4236	1.53	0.65
0.1	0.0424	1.04	0.96

With smaller updates, the weighted data changes little from one stage to the next, so consecutive stumps differ less. Each stage moves the model only a small step, and the final model is the sum of many small steps.

Figure 4 shows the effect on the circles data after 50 stages. Watch the marker sizes: with learning rate 1.0 a few points, mostly in the noisy overlap of the two classes, grow to 3.7 times their starting weight while the far points shrink; with 0.1 the largest weight is only 1.3 times the start.

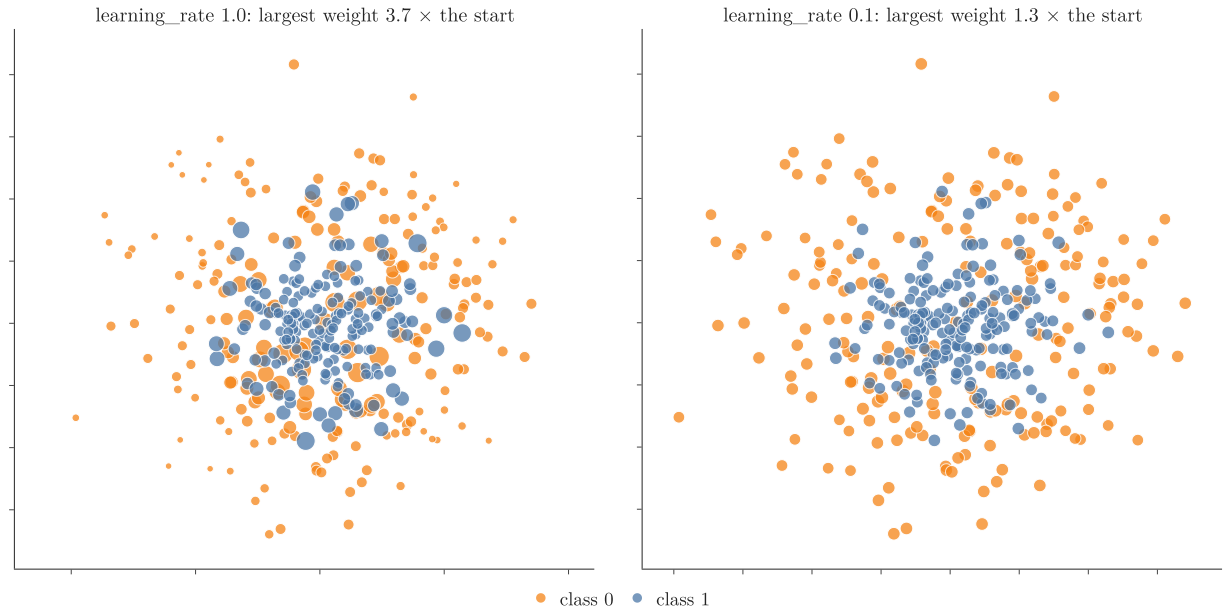


Figure 4: Observation weights after 50 stages (marker area grows with the weight), learning rate 1.0 (left) and 0.1 (right), replayed from scikit-learn's own stumps and alphas

Scaling every stump's say down this way is called **shrinkage** (G-1796) (ESL §10.12.1). Think of walking down a dark hill: short careful steps take longer but are less likely to overshoot the path. Shrinkage returns in gradient boosting.

4.3 The trade-off with n_estimators

> **Key point:** A smaller learning rate needs more stumps to reach the same fit, but tends to overfit less; the two are tuned together.

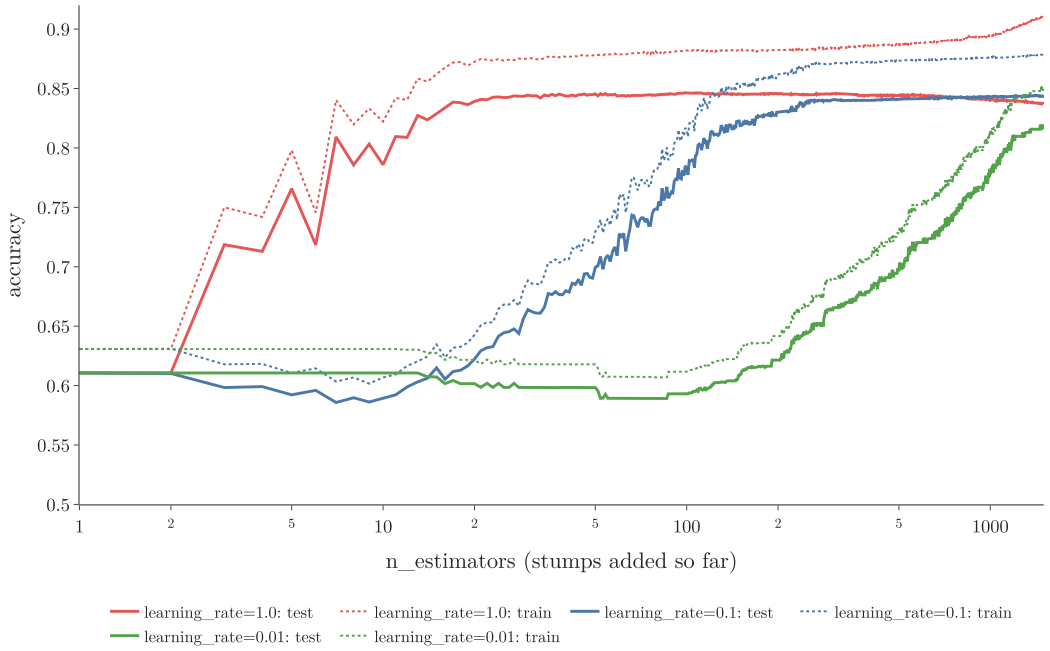


Figure 5: Training (dotted) and test (solid) accuracy as stumps are added, for three learning rates, averaged over 20 datasets; the horizontal axis is logarithmic

Figure 5 tracks accuracy as stumps are added (scikit-learn's `staged_score` (G-1868) gives the score after every stage), averaged over the same 20 datasets:

- **learning_rate 1.0** (red) learns fast: test accuracy reaches 0.846 by about 50 stumps, then slips to 0.837 by 1,500 while training accuracy keeps rising to 0.911.
- **learning_rate 0.1** (blue) needs about 150 stumps to fit the disc and is still improving at 1,500: test 0.844, training 0.878. Its train-test gap, 0.034, is half that of learning rate 1.0 at the same size (0.074).
- **learning_rate 0.01** (green) is so slow that 1,500 stumps reach only 0.819.

The smaller the learning rate, the more stumps it takes, and the less the model overfits on the way. In Figure 1, 1,500 stumps with learning rate 0.1 (bottom right) have none of the islands of 1,500 stumps with learning rate 1.0 (bottom middle).

So we do not tune the two separately. A large `n_estimators` with a small `learning_rate` is the usual recipe (ESL §10.12.1); the **grid search** (G-872) finds a good pair.

Python: n_estimators and learning_rate.

```
abc = AdaBoostClassifier(n_estimators=1500,
                         learning_rate=0.1,
                         random_state=42)
abc.fit(X_train, y_train)
abc.score(X_test, y_test)      # 0.85 on the single split
abc.estimator_weights_[:3]     # the first three alphas
```

`estimator_weights_` holds every stump's alpha after training. `staged_score(X, y)` yields the accuracy after 1 stump, 2 stumps, and so on.

5. Tuning with GridSearchCV

Key point: A grid of 4 values of n_estimators and 5 learning rates, with 10-fold cross-validation, finds 500 stumps at learning rate 0.1: accuracy 0.832, up from 0.812.

Grid search (G-872), training every combination in a list of values, was introduced in experiments: try every k, and used on a random forest in grid search over a random forest. Here the grid covers the two **hyperparameters** (G-910) that matter:

Python: Grid search over AdaBoost.

```
from sklearn.model_selection import GridSearchCV

grid = {
    "n_estimators": [10, 50, 100, 500],
    "learning_rate": [0.0001, 0.001, 0.01, 0.1, 1.0],
}
grid_search = GridSearchCV(AdaBoostClassifier(random_state=42),
                           param_grid=grid, n_jobs=-1,
                           cv=10, scoring="accuracy")
grid_result = grid_search.fit(X, y)
grid_result.best_params_
# {'learning_rate': 0.1, 'n_estimators': 500}
grid_result.best_score_        # 0.832
```

$4 \times 5 = 20$ combinations, each trained 10 times: 200 fits, about 14 seconds here. `n_jobs=-1` uses every CPU core.

Figure 6 shows all 20 scores of the grid. Watch the top two rows: with learning rate 1.0, 50 stumps are already enough (0.812); with 0.1, accuracy keeps climbing up to 500 stumps and wins (0.832). Learning rates of 0.001 and below stay at about 0.57 whatever the number of stumps, because with so small an alpha the weights barely move (section 4.2), so every new stump repeats the first one and the model stays a single straight cut, which underfits (section 3).

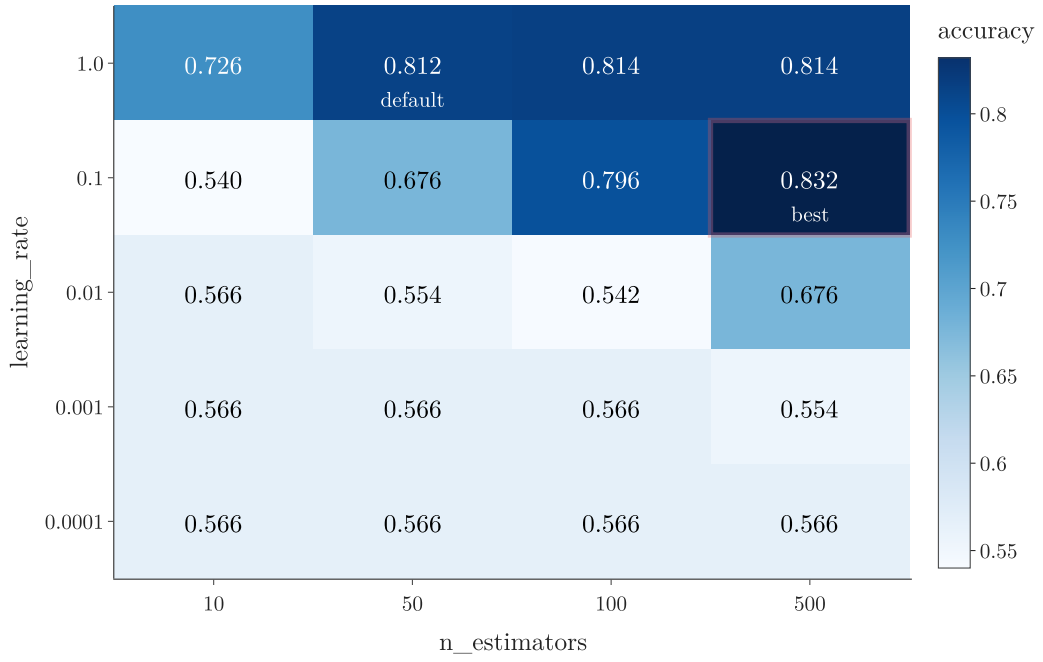


Figure 6: 10-fold cross-validated accuracy of every pair in the grid; the red box is the best pair, 500 stumps at learning rate 0.1

The best pair is **500 stumps at learning rate 0.1**, with a cross-validated accuracy of **0.832**, against 0.812 for the defaults. As section 4.3 predicted, the winner combines many stumps with a small learning rate.

Extra: The depth of the base tree can be tuned too, by listing several trees in the grid: `"estimator": [DecisionTreeClassifier(max_depth=d) for d in (1, 2, 3)]`. On the circles data the search still picks stumps (0.832). Deeper trees fit the training data faster and do worse on new data. Averaged over the 20 datasets, with 50 trees:

Base tree depth	Train	Test
1 (stump)	0.879	0.846
3	0.949	0.832
8	1.000	0.813

Deeper base trees make each learner less weak, which goes against the idea of boosting (boosting uses high-bias, low-variance models).

6. Summary

Hyperparameter	Default	What it controls	Too small	Too large
`estimator`	a stump (`max_depth=1`)	the weak learner; must accept `sample_weight`	(rarely changed)	deeper trees overfit sooner
`n_estimators`	50	number of stages (a maximum)	underfitting	overfitting, slow training
`learning_rate`	1.0	multiplier on every alpha (shrinkage)	needs many more stumps	fast learning, faster overfitting
`algorithm`	removed	(SAMME.R no longer exists)		

- `n_estimators` and `learning_rate` trade off: a smaller learning rate needs more stumps but overfits less, so the two are tuned together, not one at a time.
- The learning rate multiplies alpha, so the weight updates have a smaller amplitude and learning slows down: **shrinkage** (G-1796). Each stage then moves the model only a small step, so consecutive stumps differ less and the model fits noise more slowly.
- On the circles data, the defaults score 0.812 (10-fold cross-validation); a **grid search** (G-872) finds 500 stumps at learning rate 0.1, scoring 0.832, which is the usual recipe of many stumps with a small learning rate.
- Averaged over 20 datasets, 1,500 stumps at learning rate 1.0 widen the train-test gap from 0.033 (50 stumps) to 0.074, because each extra stage focuses on the observations still wrong, which on noisy data are mostly noise; learning rate 0.1 keeps it at 0.034.
- So of AdaBoost's few **hyperparameters** (G-910) only these two matter, and a grid search over them both is the way to tune it.

7. Sources

Built from

- CampusX, "AdaBoost Hyperparameters | GridSearchCV in Adaboost", YouTube, https://www.youtube.com/watch?v=JmXnztjULnQ

Other references

- Dietterich, T. G. (2000). "An experimental comparison of three methods for constructing ensembles of decision trees: bagging, boosting, and randomization". *Machine Learning* 40, 139–157.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning* (ESL), 2nd ed., Springer. §10.12.1, "Shrinkage".
- scikit-learn developers. AdaBoostClassifier, API reference, version 1.5, parameter `algorithm`. scikit-learn.org/1.5, page sklearn.ensemble.AdaBoostClassifier

- scikit-learn developers. Source file `sklearn/ensemble/_weight_boosting.py` (version 1.9): the early-stopping checks in `fit` and `_boost`. github.com/scikit-learn/scikit-learn

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
n_estimators (AdaBoost) (G-1290)	The largest number of simple models (weak learners) AdaBoost builds, one per boosting round (stage).
Learning rate (AdaBoost) (G-1069)	A number that scales how much each weak learner's vote (its alpha) counts in AdaBoost; values below 1 slow learning.
staged_score (G-1868)	A method that gives an ensemble's score after each added stage.
SAMME.R (G-1723)	An AdaBoost variant that used predicted probabilities; removed from scikit-learn.
Shrinkage (boosting) (G-1795)	Scaling down each base model's contribution so the ensemble learns in small steps and overfits less.
Grid search (G-872)	Training a model for every combination of listed settings and keeping the best by cross-validation.
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Decision surface (G-560)	A plot colouring every point of the input space by the class the model would predict there.
Weak learner (G-2104)	A model whose accuracy is only a little better than random guessing.
Decision stump (G-559)	A decision tree with maximum depth 1: one question, two regions. Too weak alone, it is AdaBoost's usual weak learner.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.

Term	Meaning
Underfitting (G-2035)	Being too simple to capture the pattern; fails on all data.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
max_depth (G-1184)	The cap on a tree's depth; None lets it grow until every leaf is pure.
Shrinkage (G-1796)	The pulling of a model's coefficients towards 0 by a penalty, as when Ridge's alpha grows; it makes the model rely less on any one feature and can reduce overfitting.